Reproducibility README for 'Adaptive Debiased Lasso in High-dimensional GLMs with Streaming Data'

This repository contains the source code for the **Approximated Debiased Lasso (ADL)** algorithm, designed for online statistical inference in high-dimensional generalized linear models (GLMs) with streaming data. The algorithm is particularly useful for scenarios where data arrives sequentially, and efficient, real-time inference is required.

Repository Structure

The repository is organized as follows:

Demonstrations for Reproducing Numerical Results

Below is a list of execution files for reproducing numerical results of ADL presented in **Table 2**, **Table 3**, **Figure 3** in the main text, and **Table S.5, S.10, S.13** and the left panel of **Figure 2** in the supplementary. For detailed procedures, please refer to **Section 4** of the main text and **Section S.3** of the supplementary material. Since we provide numerous simulations in Section S.3 of the supplementary material, we have included these four simulations in this repository to faciliate easier understanding and usage. These configurations are representative of the broader set of settings thereafter.

- [run_table2.py]: This execution file contains the codes necessary to reproduce the column "ADL" of Table 2 in the main text. This simulation is conducted over 500 replications with $n = 200, p = 500, s_0 = 6, \mathbf{\Sigma} = 0.1 \times \{0.5^{|i-j|}\}_{i,j=1,\ldots,p}$. The confidence intervals are constructed for three randomly selected parameters from each category of $\boldsymbol{\beta}^*$.

- [run_table3.py]: This execution file contains the codes necessary to reproduce the column "ADL" of Table 3 in the main text. This simulation is conducted over 500 replications with $n = 200, p = 500, s_0 = 6, \mathbf{\Sigma} = \{0.5^{|i-j|}\}_{i,j=1,\ldots,p}$. The confidence intervals are constructed for three randomly selected parameters from each category of $\boldsymbol{\beta}^*$.

- [run_figure3.py]: This execution file contains the codes necessary to reproduce Figure 3 in the main text. In this demonstration, the dimension p is raised to 20000 with $n = 1000, s_0 = 20, \mathbf{\Sigma} = \{0.5^{|i-j|}\}_{i,j=1,\ldots,p}$. The confidence intervals are constructed for three randomly selected parameters from each category of $\boldsymbol{\beta}^*$.

- [run_tableS5.py]: This execution file contains the codes necessary to reproduce the column "ADL" of Table S.5 (multiple levels of signal strength) in the supplementary material. This simulation is conducted over 500 replications with $n = 200, p = 500, s_0 = 10, \mathbf{\Sigma} = \{0.5^{|i-j|}\}_{i,j=1,\ldots,p}$. In this case, the non-zero coefficients are equally selected from $\{0.2, 0.4, 0.6, 0.8, 1.0\}$

- [run_figureS2.py]: This execution file contains the codes necessary to reproduce the left panel of Figure 2 (power curve) in the supplementary material. This figure shows power curves of Wald test for a single coefficient under the first setting in Scetion S.3.2, where $p = 500, s_0 = 6$, with varying sample size and $\mathbf{\Sigma} = \left\{0.5^{|i-j|}\right\}_{i,j=1,\ldots,p}$. Simulation results are averaged over 2000 replications.

- [run_tableS10.py]: This execution file contains the codes necessary to reproduce the columns "AD+MCP" and "AD+SCAD" of Table S.10 (different penalty functions) in the supplementary material. In this simulation, we explore the performance by replacing the lasso with SCAD (Fan and Li, 2001) and MCP (Zhang, 2010). Details can be found in Section S.3.4 of the supplementary material. Note that the column "ADL" is exactly the same as Table 3 in the main text, since we have fixed the random seeds for this study. This simulation is conducted over 500 replications with $n = 200, p = 500, s_0 = 6, \mathbf{\Sigma} = \{0.5^{|i-j|}\}_{i,j=1,\ldots,p}$.

- [run_tableS13.py]: This execution file contains the codes necessary to reproduce Table S.13 (Poisson regression) in the supplementary material. This simulation is conducted over 500 replications with $n = 200, p = 500, s_0 = 6, \mathbf{\Sigma} = 0.08 \times \{0.5^{|i-j|}\}_{i,j=1,\ldots,p}$. In this simulation setup, we set half of the nonzero elements of $\boldsymbol{\beta}^*$ to be 0.4 and the other half to be -0.4. Details of the simulation setup can be found in Section S.3.5 of the supplementary material.

Real Data Example

- [run_realdata.py]: Execution file for real data example. Please read through **Real Data Example** section in this README file for detailed procedures.

Algorithm Core Functions

- [adl.py]: This file implements the **Approximated Debiased Lasso (ADL) algorithm**, the main method proposed in the paper for online statistical inference in high-dimensional GLMs.

- [adl_realdata.py]: This file implements the ADL algorithm for real data analysis, which is compatible with sparse arrays.

- [radar.py]: This file contains the implementation of the **Regularization Annealed Epoch Dual Averaging (RADAR)** and **Adaptive RADAR algorithms**, which are core components of the ADL algorithm.

Helper Functions

- [cal.py]: This file contains utility functions for generating synthetic data, calculating summaries, and visualizing results.

- [process.py]: This script processes the raw dataset (`combined_data.csv`) to extract uni-gram and bi-gram features from text data, and transforming it into a sparse matrix format for efficient storage and analysis.

Dependencies

To reproduce the numerical results, we suggest install the the following dependencies in your Python environment:

- `numpy` , version 2.2
- `scipy` , version 1.15
- `matplotlib` , version 3.10
- `pandas` , version 2.2.3

Reproducing Numerical Results

To run the simulations included in this repository, execute the following files:

```
python run_table2.py
python run_table3.py
python run_figure3.py
python run_tableS5.py
python run_figureS2.py
python run_tableS10.py
python run_tableS13.py
```

For each simulation, the online estimates, confidence intervals, and trace plots will be saved in a dedicated folder. For example, the results for Table 2 will be saved in the folder `./table2`. For the ease of presentation, the summaries required for reporting in the manuscript are saved in a text file named `summary.txt` within the corresponding folder.

Real Data Example

Dataset

For the real data example, the dataset `combined_data.csv` is required. This dataset can be downloaded from:

- Email Spam Classification Dataset on Kaggle

To extract uni-gram and bi-gram features from the raw data, ensure that the raw dataset is placed in the root directory of the repository before running the real data analysis scripts. Then, execute the following command:

```
python process.py
```

The processed data will be saved as sparse matrices in `bigram_X.npz` and `bigram_y.npz` for further analysis. For convenience and to facilitate an easier walkthrough of the code, we have included these two processed data files in the repository. Users may skip the feature extraction step and proceed directly to online inference if desired.

As described in Section 5 of the main text, we selected three terms of interest: “investment”, “schedule”, and “per cent” for statistical inference. These terms correspond to feature indices 6795, 7856, and 22608, respectively. Users can specify which feature to analyze by modifying line 13 of the script file `run_realdata.py` .py. The trace plot and test prediction error will be saved in a folder, for example, `./realdata_result/feature6795`. Online estimates and confidence intervals will also be

saved in the corresponding folder. To conduct online statistical inference on the processed data, run the following script:

```
python run_realdata.py
```